Trabajo Práctico 1 | Gradiente de Política

Carlos Federico Trejo • I-404 Aprendizaje Reforzado • Universidad de San Andrés • Sep. 2026

1. Introducción

En este trabajo se implementó el algoritmo REINFORCE (gradiente de política) y se lo usó para entrenar agentes en seis entornos distintos: cuatro armados especialmente para poder debuggear el algoritmo y dos que ya vienen con Gymnasium (CartPole-v1 y Acrobot-v1), más exigentes. Después se extendió el algoritmo agregándole un baseline (una red V que estima cuánto retorno se espera desde cada estado) para tratar de reducir la varianza del entrenamiento. En este informe se cuenta cómo se armó, qué resultados dio, y qué cosas se aprendieron que no se esperaban de antemano.

2. Desarrollo REINFORCE

La política se representa con una red neuronal chica (dos capas ocultas de 20 neuronas con activación ELU) que recibe el estado y devuelve un logit por cada acción posible. Para elegir la acción se arma una distribución categórica a partir de esos puntajes (con `torch.distributions.Categorical`) y se muestrea de ahí, porque la política tiene que ser estocástica para poder explorar durante el entrenamiento. Para entrenar, se junta un batch de trayectorias con la política actual, se calcula el retorno descontado de cada paso (con $\gamma = 0{,}99$ en todos los entornos) y esos se usan para armar el gradiente que se optimiza con Adam. Antes de actualizar la política se normalizan los retornos del batch (media 0, desvío 1), como recomienda el enunciado, para que el entrenamiento no sea tan ruidoso.

3. Para qué sirve cada entorno

TwoAZeroObsOneStep. Es el caso más simple, un solo estado y un solo paso por trayectoria. Sirve para probar lo más básico de REINFORCE, que el update realmente aumente la probabilidad de la acción que da mejor recompensa.

TwoARandomObsOneStep. Acá la acción correcta depende de en qué estado arrancó la trayectoria (S1 o S2). Sirve para confirmar que la red está usando la observación para decidir y no aprendiendo una acción fija. Una política que ignore el estado no puede resolver este entorno, en promedio le iría mal la mitad de veces.

LineWorldEasyEnv. La recompensa aparece recién al final del pasillo, sirve para probar que el algoritmo propaga bien la recompensa hacia atrás, es decir que el cálculo de los retornos descontados (con el factor γ) está bien hecho. Si el descuento estuviera mal calculado, el agente tendría problemas para darse cuenta de que moverse a la derecha desde el principio también conviene aunque el premio esté más adelante.

LineWorldMirrorEnv. En este entorno ninguna política que siempre haga lo mismo puede resolverlo porque en el medio del pasillo las acciones se invierten. Sirve para confirmar que el agente aprende una política que realmente depende del estado a lo largo de varios pasos, combinando lo que se prueba en TwoARandomObsOneStep (usar el estado) con lo que se prueba en LineWorldEasyEnv (varios pasos seguidos).

4. Resultados de entrenamiento

Cada experimento se corrió con cinco semillas y todo lo que se reporta es el promedio. Los seis entornos usan el mismo presupuesto, 150 iteraciones de un batch de 16 trayectorias, o sea 2400 trayectorias cada uno. En los cuatro entornos chicos el learning rate de la política se fijó en 0,01; para CartPole-v1 y Acrobot-v1 se eligió con un grid search chico sobre 0,01, 0,005 y 0,001: se hace un entrenamiento corto de quince iteraciones por combinación de candidato y semilla, y gana el candidato con mejor retorno promedio. Las líneas de referencia se estiman corriendo cada agente fijo 200 trayectorias en los entornos chicos y 50 en los dos de Gymnasium.

En la figura 1 están los seis entornos, una fila cada uno: a la izquierda la curva de recompensa (o de pasos por trayectoria en los dos pasillos, donde lo que importa es llegar en menos pasos) con las líneas de referencia del agente aleatorio y de los agentes triviales (acción constante), y a la derecha la curva de pérdida. Las cinco semillas van en tono claro y por encima el promedio en tono grueso con una media móvil de diez.

En los cuatro entornos chicos el agente supera claramente al aleatorio: el retorno llega a 1 en los de un solo paso y los pasos caen al mínimo en los pasillos. Contra los triviales la lectura cambia según el entorno. En LineWorldEasyEnv la política termina empatando con "siempre a la derecha" (5 pasos), porque esa acción ya es óptima. En TwoARandomObsOneStep ninguna acción constante puede superar al azar, porque la acción correcta cambia con el estado: las dos valen 0 en promedio y en la evaluación dan $\pm 0{,}11$, que es el ruido de esa estimación. El agente, en cambio, llega a 1. En LineWorldMirrorEnv no hay líneas triviales: ninguna acción constante llega a la meta. CartPole-v1 alcanza el techo de 500 pasos y Acrobot-v1 pasa de -500 a alrededor de -90.

En REINFORCE la pérdida no compara la salida de la red contra una respuesta correcta: lo que se busca es aumentar el retorno esperado, y como los optimizadores minimizan, se les pasa el negativo de ese objetivo. Ese número no tiene por qué bajar para que el agente mejore. La pérdida es, en cada paso, el logaritmo de la probabilidad que la política le asignó a la acción tomada, multiplicado por el retorno normalizado de ese paso. En los cuatro entornos chicos el agente termina eligiendo siempre la misma acción en cada estado, con probabilidad casi 1, y el logaritmo de 1 es 0: el producto se anula y la curva queda pegada al cero. En CartPole y Acrobot la política nunca llega a esa certeza, porque los estados son continuos y siempre aparecen situaciones nuevas, así que ese logaritmo se mantiene lejos de cero y la pérdida sigue oscilando. Para saber si el agente aprende hay que mirar el gráfico de la izquierda.

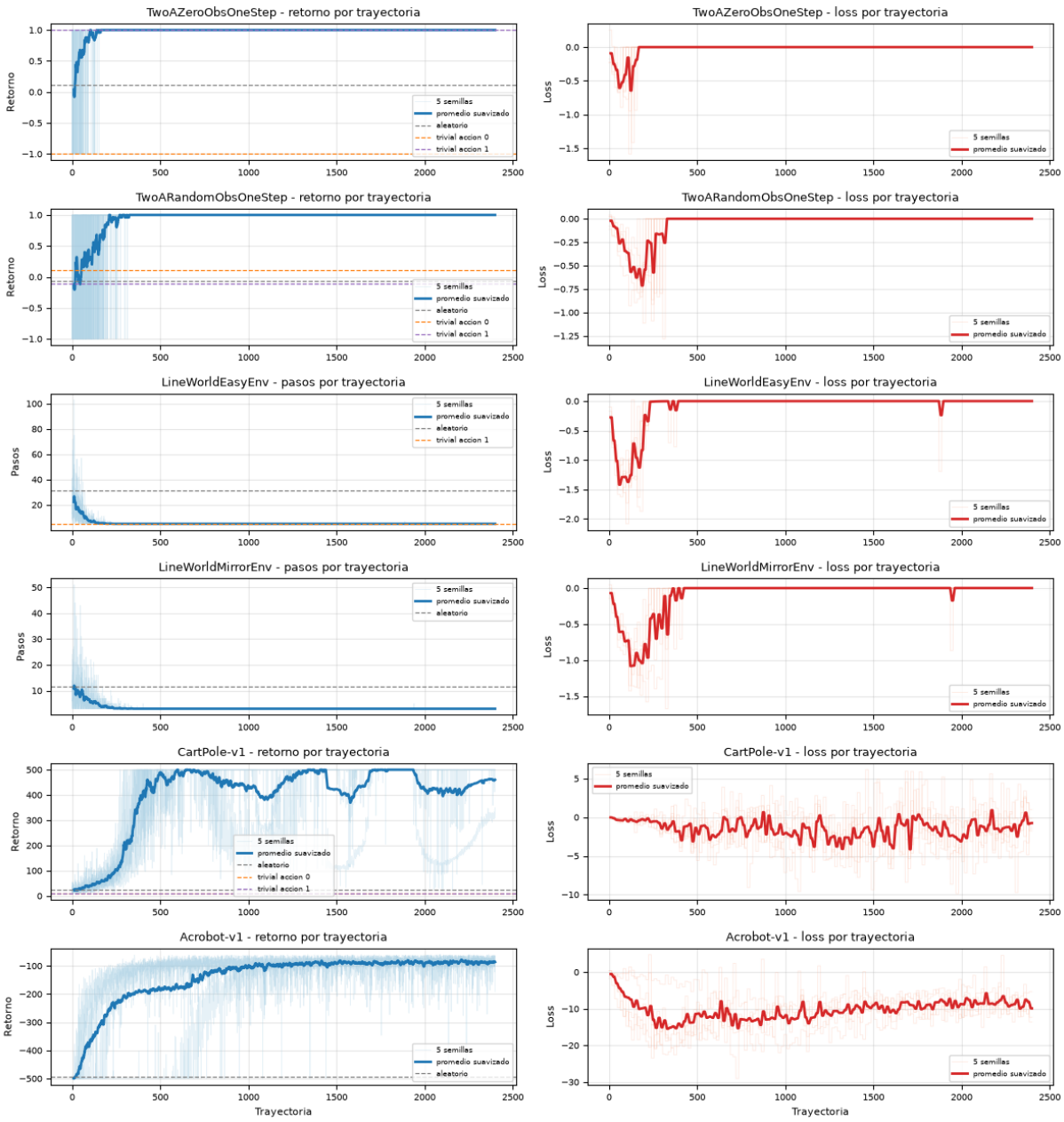


Figura 1: Los seis entornos entrenados con REINFORCE. Una fila por entorno: a la izquierda retorno por trayectoria (o pasos por trayectoria en los dos *LineWorld*) con las líneas de referencia, a la derecha la pérdida. Las líneas finas claras son las cinco semillas sin suavizar y la gruesa es el promedio suavizado.

5. El entorno que más costó

Si se considera que converger es llegar al nivel final, el más lento es Acrobot-v1, cuyo promedio sigue subiendo hasta cerca de la trayectoria 1000. Si converger es quedarse ahí, el más lento es **CartPole-v1**, que no se asienta nunca. Llega al techo de 500 cerca de la trayectoria 500, pero después se desploma tres veces.

Acrobot-v1 es lento al inicio: hasta el primer balanceo exitoso todas las trayectorias fracasan igual y dan el mismo retorno, y como el algoritmo mueve cada acción según cuánto mejor o peor le fue que al promedio del batch, si todas salen iguales no hay nada que mover. CartPole-v1 es lento al final: llega al techo rápido pero no lo sostiene. Ahí todas las trayectorias duran 500 pasos, así que lo que hace que un paso valga más que otro no es la posición del carro sino cuántos faltan para el corte y eso no está en la observación: dos momentos con el carro igual de equilibrado valen muy distinto según ocurran al principio o al final. Esa varianza que el estado no puede explicar es la que produce los desplomes y es lo que se buscará corregir con un baseline.

6. Qué fue lo más demandante

Lo más demandante no fue programar el algoritmo sino descubrir que la red de valor de la Parte 4 nunca estaba aprendiendo, algo que se podría haber evitado midiendo desde el principio si el baseline reducía la varianza del estimador, en vez de mirar solamente si el retorno final mejoraba. Más sobre esto en la sección 7.1.

7. Comparación con baseline

El baseline se implementó como una segunda red que recibe el estado y devuelve una estimación de $V(s)$.

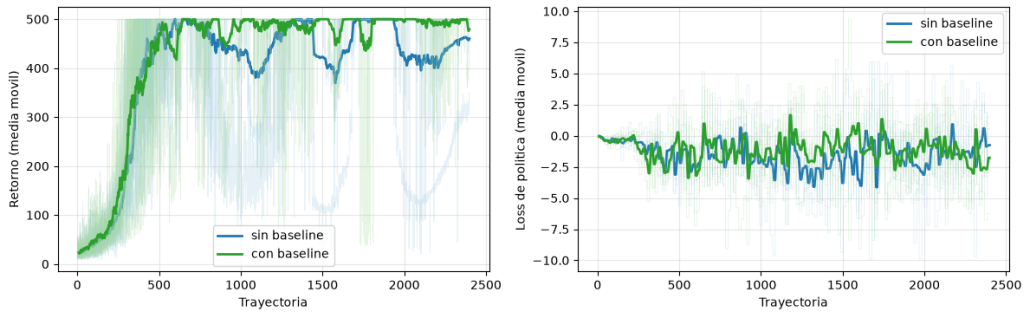


Figura 2: CartPole-v1 con baseline (Parte 4) y sin baseline (Parte 3). Las líneas finas son las cinco semillas.

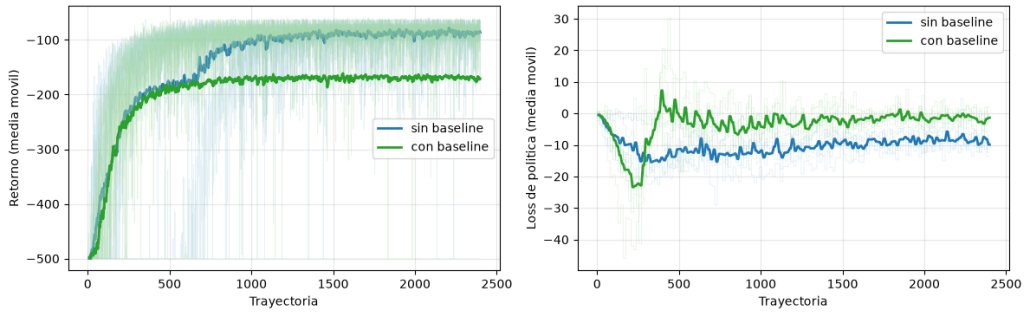


Figura 3: Acrobot-v1 con baseline (Parte 4) y sin baseline (Parte 3). Una de las semillas con baseline no aprende y baja el promedio.

Con eso se arma la ventaja $A(t) = R(t) - V(s_t)$, que reemplaza al retorno en la actualización, y la red de valor se entrena aparte por regresión contra el retorno.

El learning rate de la política se elige una sola vez en la Parte 3 y la Parte 4 lo hereda (0,01 en CartPole y 0,005 en Acrobot). Si cada versión usara el suyo, la diferencia medida mezclaría el efecto del baseline con el de la escala del gradiente, que restar $V(s)$ también cambia. Del grid search de la Parte 4 sale solo el learning rate de la red de valor, que no afecta al gradiente de la política: se prueban 0,01, 0,05 y 0,1, con un entrenamiento de cuarenta iteraciones por cada combinación de candidato y semilla, y se elige con el criterio que se explica en la sección 7.1.

Cabe destacar que la Parte 2 usaba $(R - \bar{R})/\sigma_R$ y la Parte 4 usa $(A - \bar{A})/\sigma_A$. Restar la media del batch ya es en sí un baseline, así que la comparación no es entre no tener baseline y tener, sino entre un baseline constante y uno que depende del estado. Dividir por el desvío reescala el gradiente y funciona como ajuste del learning rate, pero está en las dos versiones, así que no afecta la comparación.

7.1. Verificar que el baseline actúe

Al revisar el desempeño con y sin baseline en el gráfico de retorno, se detectó que no hubo una mejoría clara, entonces se decidió a investigar más a profundidad este tema. El retorno final no permite saber si el baseline funciona, porque un baseline que no ayuda y uno que directamente no está actuando se ven igual. Por eso se midió su efecto de forma directa.

Un baseline no cambia hacia dónde apunta el gradiente, solo su varianza, y la cantidad afectada es la que multiplica al logaritmo de la probabilidad: el retorno $R(t)$ sin baseline y la ventaja $A(t)$ con baseline. Se registró en cada iteración el desvío de las dos sobre el mismo batch, con la red de valor tal como estaba antes de ese paso, y se reporta el cociente σ_A/σ_R, que vale 1 si el baseline no resta nada y baja cuanta más varianza saca. Ese mismo cociente se usó como criterio para elegir el learning rate de la red de valor.

Con el learning rate elegido en un primer momento (0,01) el cociente dio 1 en los dos entornos: el baseline no restaba nada, y la red de valor se quedaba en cero mientras el retorno promedio valía alrededor de 80. Con 0,1 el cociente bajó a 0,88 en CartPole-v1 y 0,78 en Acrobot-v1. La pérdida de la red de valor no alcanzaba para detectar el problema, porque también bajaba con 0,01: solo que se estancaba cuatro veces más arriba.

7.2. Resultados

Esa menor varianza del estimador se traduce en corridas más parecidas entre sí. En CartPole-v1 la varianza del resultado final entre las cinco semillas cayó de 111,9 a 4,1. Sin baseline hay corridas que se desploman a mitad de camino, con baseline las cinco quedan pegadas al techo de 500. Es el entorno donde la varianza era el problema, así que es donde más se nota.

En Acrobot-v1 el gráfico muestra menos recompensa con baseline que sin él. La causa no es que la política sea peor: cuatro de las cinco semillas llegan a -90, igual que sin baseline, y la quinta no despega nunca de -500. Esa sola arrastra el promedio hasta -172, un valor en el que no terminó ninguna corrida.